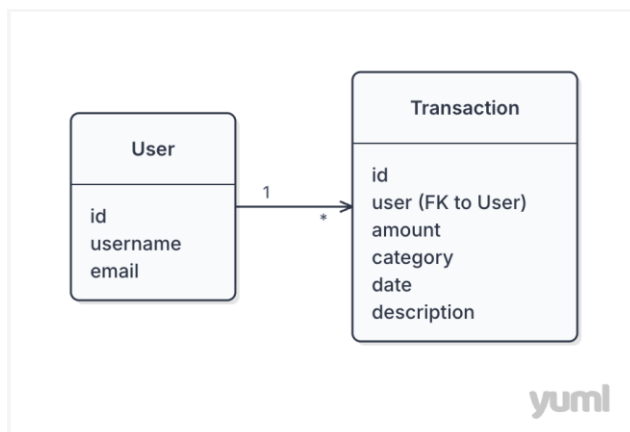


transactions/views.py

1. The CRUD features are handled through the four views: `transactions_page`, `transaction_create`, `transaction_update`, and `transaction_delete`. Each one covers the basic operations as expected.
2. All queries are limited to the logged-in user. This is done with things like `filter(user=request.user)` and `get_object_or_404(..., user=request.user)`, so users only see their own data.
3. For deleting a transaction, the view first shows a confirmation page on a GET request, and the actual deletion only happens on POST.
4. Redirects and template names are kept consistent across the views, mainly using `redirect("transactions:page")` to keep navigation simple.

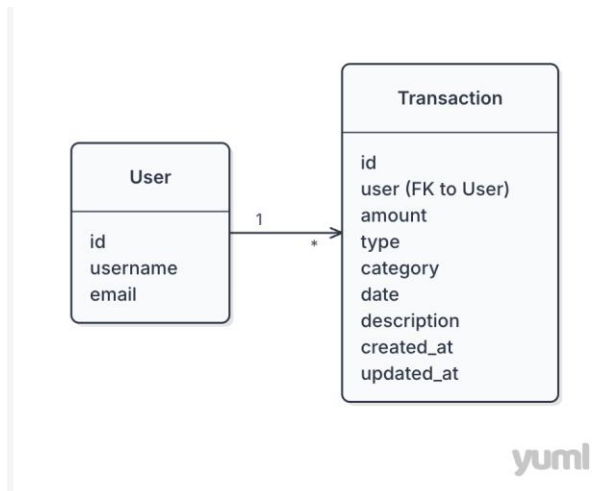


transactions/models.py

1. The model includes the required fields: `amount`, `category`, `date`, and `description`. These match the prompt directly.
 - o `amount = models.DecimalField(...)`
 - o `category = models.CharField(...)`
 - o `date = models.DateField()`
 - o `description = models.CharField(max_length=255, blank=True)`
2. Categories are separated into "income", "fixed", and "variable" so the system can clearly distinguish between different types of expenses and income.

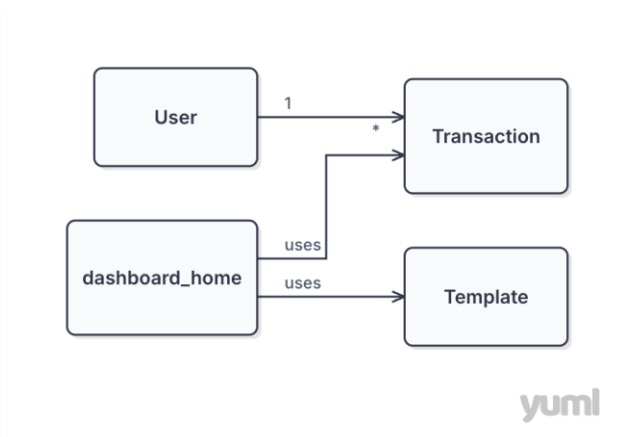
Additional parts that weren't specifically asked for but make sense in the model:

- A `user` foreign key connects each transaction to the correct account.
- A `type` field (income/expense) gives another way to classify entries.
- `created_at` and `updated_at` timestamps help track changes.
- The default ordering shows the newest transactions first.



dashboard/views.py

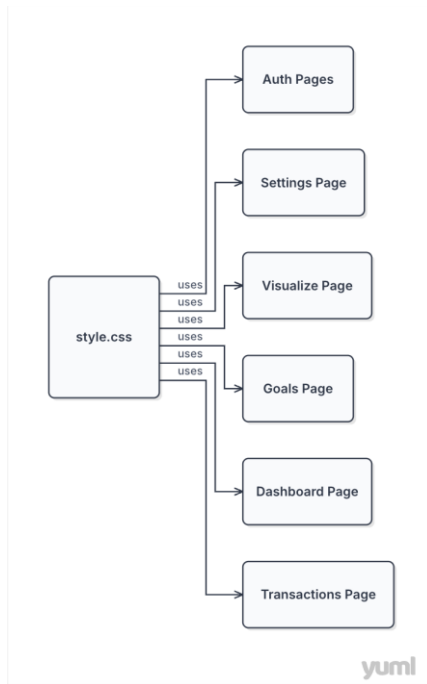
1. The dashboard calculates total income, total expenses, and the remaining balance using Django's ORM. It filters by the current user and uses `.aggregate(Sum("amount"))` for the totals.
2. Recent transactions are taken with `transactions[:5]`. For monthly data, the view filters by the current month and uses `TruncMonth` to group entries. After that, the results are turned into lists (`chart_labels`, `income_data`, `expense_data`) so they can be used in charts.



static/css/style.css

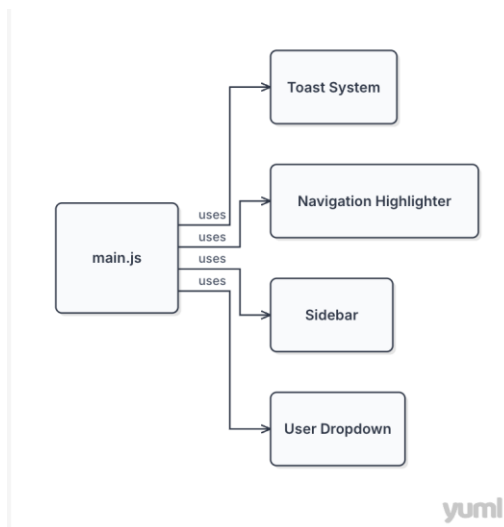
1. The project no longer uses Bootstrap. Everything is styled with custom CSS, including variables like `--color-*` and layout systems using Flexbox and Grid. Buttons, cards, tables, forms, and other UI parts are all written manually.
2. All pages load the same CSS file, so the design stays consistent everywhere. The file includes resets, layout rules, and shared components.

- Layout issues and scaling problems were adjusted using Grid/Flexbox. Breakpoints at 1100px and 768px handle responsiveness. Spacing, shadows, and typography are kept consistent across the UI.



static/js/main.js

- Dark mode and localStorage support are not included yet. There's no theme switching or saved preferences in the current version. The script mainly handles navigation highlighting, dropdown menus, sidebar toggling, and toast messages.
- Chart.js isn't implemented yet, but the file is structured in a way that makes adding it later straightforward. Everything is wrapped inside `DOMContentLoaded`, and helper functions like `showToast` are already set up.



accounts/views.py

1. Login, registration, and logout use Django's built-in authentication system.
 - o Login uses `authenticate()` and `login()`.
 - o Registration validates the form and creates a new user, then logs them in.
 - o Logout uses Django's `logout()` and redirects to the login page.
2. A user settings page (for updating username or password) hasn't been added yet. The current setup could support it later, but it's not implemented in this version.
3. Validation and error handling rely on Django's messages framework and form errors. Invalid login attempts show an error message, and registration errors appear directly in the form.

